Take-Home Assignment — Head of Platform Engineering (Principal Engineer)

Estimated effort: 3–4 days **Stack:** Your choice — **Java + Spring Boot** *or* **Python + FastAPI/ Django Submission:** Git repository (link or zip) + written design document

---

1. Before you start — what this assignment is and isn't

This is a Principal-level exercise, so we care far more about **architecture, judgment, and the reasoning behind your decisions** than about how many features you can cram in. A smaller, well-built, well-explained system will always beat a sprawling, half-finished one.

Two things we are explicitly **not** testing:

- **Production polish.** We don't expect a hardened, fully-tested, deployable platform in a few days. We expect a credible core, plus a clear account of what you'd do differently with more time and at real scale.
- **Working in isolation.** AI coding assistants (Claude Code, Copilot, Cursor, ChatGPT, etc.) are a normal part of how we build at HBox. **Use them freely** — we'd actually like to see how you work with them (see Part D).

If anything in this document is ambiguous, make a reasonable assumption, **write it down**, and move on. How you handle ambiguity is part of what we're evaluating.

---

2. The scenario

You're joining **VitalLink Health** (a fictional company for this exercise, broadly similar to HBox). VitalLink ships connected medical devices — blood-pressure cuffs, glucometers, pulse oximeters, weight scales — to patients enrolled in **Remote Patient Monitoring (RPM)** programs. Devices transmit readings to our cloud throughout the day. **Care coaches** monitor those readings and intervene when something looks wrong.

Today the platform is a single overloaded monolith. Reading ingestion stalls during morning peaks (most patients measure between 6–9 AM), alerts are evaluated synchronously inside the ingest path, and there's no clean separation between intake and the rest of the system.

You've been asked to **carve out the Readings & Alerting capability into a standalone service** that can scale independently and become a template for how the rest of the platform is rebuilt.

Domain model (simplified)

- A **patient** is enrolled in one or more programs and has a set of **devices**.
- A **device** emits **readings** (e.g. systolic/diastolic + pulse for a BP cuff; mg/dL for a glucometer). Devices are intermittently connected and **retry**, so the same reading can arrive more than once.
- Each patient has **thresholds** per metric (e.g. systolic > 180 or < 90 = critical). A reading that breaches a threshold should generate an **alert**.

- A **care coach** is assigned a panel of patients and works from a queue of alerts.

---

3. Part A — Core implementation *(required)*

Build a **Readings & Alerting microservice**. Functionally it must:

1. **Ingest readings** via a REST endpoint that accepts both single readings and **batches** (devices buffer offline and flush on reconnect).
2. **Authenticate devices** — each request is tied to a device/patient identity (device API key or JWT; your call). Reject unauthenticated or mismatched device/patient pairs. If you'd use a different scheme for service-to-service or care-coach access (e.g. OAuth2 / OIDC), note it — you can implement one scheme and describe the rest in Part B.
3. **Handle duplicates idempotently.** A device retrying the same reading must not create duplicates or re-fire alerts. Design this deliberately — it's a key senior signal.
4. **Persist** readings to a relational store (PostgreSQL or MySQL) with a schema you've thought about for **high write volume and time-series access patterns**. Indexing/partitioning choices matter.
5. **Evaluate thresholds and raise alerts** — but do the alert work **asynchronously** (via Kafka / RabbitMQ / SQS, or a comparable queue). Ingestion should stay fast and not block on alert processing.
6. **Process alerts in a worker** that consumes from the queue and records a notification for the relevant care coach (logging/persisting a notification is fine — you don't need to integrate a real notifier).
7. **Cache** appropriately with Redis — e.g. latest reading per patient and/or threshold lookups on the hot ingest path. Be explicit about your invalidation strategy.
8. **Expose read APIs:** - Latest readings for a patient (and a time-range query). - The alert queue for a given care coach.
9. **Show it under a little load.** Include a small script (k6, Locust, a `curl` /async loop — whatever you like) that fires a burst of batched readings, and report the ingest throughput and rough latency you observe locally in your README. We're not benchmarking hardware; we want to see that you measured, and that you can reason about where the bottleneck is.

Must run with one command

Provide a `docker-compose` (or equivalent) that brings up the service, the database, the cache, and the message broker so we can run it locally without hand-assembly. Include seed data and a couple of example `curl` requests (or a small script) that demonstrate the happy path and the duplicate-handling path.

What "good" looks like here

Clean separation of concerns and clear architecture — sensible use of OOP and design patterns where they earn their place (not ceremony for its own sake) — a sensible API contract, a schema that won't fall over at volume, correct and intentional concurrency/idempotency handling, and tests

on the parts that matter most (we'd rather see a few high-value tests than blanket coverage). You do **not** need to implement every metric type or build a UI.

---

4. Part B — System design write-up *(required)*

This is weighted as heavily as the code. In a document (Markdown or PDF, ~2–4 pages plus a diagram), address:

- **Architecture diagram** of the target-state system at scale (a clear sketch is fine — draw.io, Excalidraw, Mermaid, whiteboard photo).
- **Scaling to ~1M active devices and ~50M readings/day, with sharp morning bursts.** Where does your Part A design break, and what changes? Cover throughput, the ingest path, and the alerting pipeline.
- **Data at scale:** partitioning/sharding, retention/archival of time-series vitals, and how reads stay fast as the table grows.
- **Correctness under stress:** out-of-order arrivals, late/duplicate data, backpressure when the broker or DB is slow, and what happens during a partial outage.
- **Healthcare realities:** this is PHI. Discuss security, encryption, access control, auditability, and what HIPAA-style compliance implies for your design (at the level of architectural decisions, not legal text).
- **Observability:** what you'd instrument to know the system is healthy.
- **A live incident.** It's 7:15 AM. Ingest p99 latency has tripled in ten minutes, the alert queue is backing up, and care coaches are paging you about stale data. Walk us through your diagnosis: what you look at first, the likely causes in *this* architecture and how you'd tell them apart, what you'd do to stabilize it now, and what you'd change afterward so it doesn't recur. We want the reasoning of someone who has carried a pager, not a generic checklist.
- **Tradeoffs:** call out where your Part A implementation differs from what you'd ship to production, and why those shortcuts were the right call for a 3–4 day exercise.

---

5. Part C — AI integration *(required, but keep it light)*

Pick **one** and implement it as a small, well-considered feature. We're evaluating **integration judgment**, not model sophistication — robustness, latency/cost awareness, and graceful failure matter more than clever prompting.

- **Reading summary:** an endpoint that uses an LLM to produce a plain-language summary of a patient's recent readings for a care coach ("BP trending up over the last 4 days, two critical spikes…").
- **Alert triage:** an LLM-assisted classifier that proposes a severity/priority for an alert, with the model's output treated as advisory rather than authoritative.

Whichever you choose, be explicit about: timeouts and fallbacks (the LLM must never block or break the ingest/alert path), cost/rate-limit handling, how you'd evaluate output quality, and the failure

modes you considered. A stubbed/mocked LLM client is acceptable if you don't want to wire up a real key — what we want to see is the **integration pattern**.

---

6. Part D — How you worked *(short, required)*

A few paragraphs in your README: which AI tools you used and for what, where they helped, and where you had to override or correct them. We're not looking for a "right" answer — we want insight into how you operate as an engineer who leads with these tools.

---

7. Part E — Technical leadership *(short, required)*

This is a Head-of-Platform role, so we want a window into how you'd raise the bar for a team — not just your individual output. Two short pieces, both in your README or design doc:

- **Onboarding note (½–1 page).** Write the "design decisions & conventions" section of your README as if you're handing this service to a mid-level engineer who will own and extend it. Spell out 2–3 conventions you'd hold the line on in code review (naming, error handling, testing boundaries, API versioning — your pick) and *why* each one matters here. We're looking for the judgment that prevents a codebase from rotting, communicated in a way a teammate can actually act on.
- **A code-review take.** Point to one place in your own submission where you made a deliberate tradeoff or knowingly cut a corner, and write the review comment you'd want a senior engineer to leave on it. This tells us how you think about quality when you're the one being reviewed.

Keep it concise — depth of reasoning beats length.

---

8. Stretch goals *(optional — won't count against you if skipped)*

Only if you have time and interest. Pick at most one or two; depth beats breadth.

- A rate-limiting / backpressure mechanism on ingest with a load-test demonstrating it.
- An at-least-once → effectively-once story for alert delivery, demonstrated.
- A read-replica / CQRS split for the query APIs.
- A brief migration plan for cutting the existing monolith over to this service with zero downtime.

---

9. Ground rules

- Use any libraries, frameworks, and AI tools you like.
- Keep secrets out of the repo; use env vars / example config.
- Commit incrementally — we like to see how the work evolved.

---

10. Deliverables checklist

☐ Git repo (link or zip) with the Part A service.

☐ `docker-compose` (or equivalent) + seed data + example requests that run locally.

☐ A small load script + the throughput/latency numbers you observed (Part A #9).

☐ Tests on the parts that matter most.

☐ `README.md`: setup/run instructions, key design decisions, assumptions, the Part D write-up, and the Part E leadership pieces.

☐ System design document (Part B) with an architecture diagram, including the live-incident walkthrough.

☐ The Part C AI feature, with its rationale documented.

11. How we'll evaluate

We score against the dimensions below. Code quality matters, but for a Principal/Head role, **architecture, judgment, leadership, and communication carry the most weight.**

Dimension	What we look for	Weight
System design & scalability	Sound target-state architecture; realistic scaling plan; thoughtful data modeling and partitioning; handles ordering/duplicates/ backpressure; credible incident diagnosis	30%
Code quality & correctness	Clean architecture, sensible API design, correct idempotency & concurrency, meaningful tests, runs as documented	25%
Engineering judgment & tradeoffs	Right-sized decisions, clearly justified; knows what to skip and why; security/PHI awareness	15%
Technical leadership	Conventions worth enforcing, communicated so a teammate can act on them; honest self-review; raises the bar beyond individual output	10%
Communication	README and design doc are clear, well-organized, and let a reader follow the reasoning	10%
AI integration & usage	Robust, cost/latency-aware integration; honest, insightful account of how AI tools were used	10%

12. FAQ

Do I have to use both Java and Python? No — pick the one stack you're strongest in. The role spans both, so use whichever lets you show your best work.

Can I use a managed cloud service instead of running a broker/cache locally? Local via docker-compose is preferred so we can run it without credentials. If you'd architect it on a managed service in production, say so in Part B.

How complete does Part A need to be? Complete enough to demonstrate the end-to-end path — ingest → dedup → persist → async alert → notification → read APIs — for at least one metric type. Breadth across every device type is not required.

Can I ask questions before starting? Yes — reply to this email and we'll get back to you quickly. Reasonable assumptions documented in your README are also completely fine.

We're looking forward to seeing how you think. Good luck.